

Sferična aberacija sferičnih leč in raytracing

Seminarska naloga pri predmetu Numerične metode

Avtor: Taj Šobot

Mentor: doc. dr. Rene Markovič

June 24, 2026

Kazalo

1	Uvod	2
2	Fizikalno ozadje	2
2.1	Snellov zakon	2
2.2	Vektorska oblika Snellovega zakona	2
2.2.1	Izpeljava	2
2.2.2	Popolni odboj	3
3	Geometrija sfere in žarka	3
3.1	Enačba sfere	3
3.2	Presečišče žarka s sfero	3
3.3	Normala v točki presečišča	4
4	Opis programa	4
4.1	OpenGL in shaderji	4
4.2	Compute shader	4
4.3	Metanje žarkov	4
4.4	Zgradba scene	5
5	Implementacija v GLSL	5
5.1	Compute shader — presečišče s sfero	5
5.2	Compute shader — vektorski lom	5
5.3	Potek žarka skozi lečo	5
5.4	Vzorčenje slike na zaslonu	6
6	Parametri programa	6
7	Rezutati	7
7.1	Testne slike	7
7.2	Prvi render	8
7.3	Drugi render	9
7.4	Tretji render	10
7.5	Četrty render	11
7.6	Peti render	12
8	Zaključek	13
9	Viri	14

1 Uvod

Sferična aberacija je ena izmed optičnih napak, ki nastane, ko leča ne lomi vseh vpadnih žarkov v isto gorišče. V tej nalogi smo analizirali lom žarkov skozi sferično lečo, kjer je ta pojav bistveno opazljiv. Bolj oddaljeni so vpadni žarki od optične osi leče, večji pogrešek imajo. Rezultat je popačena slika. V inštrumentih kot so kamere in teleskopi, kjer nastopajo sistemi leč pa to prevede v zamegljene in ne ostre slike.

Lečo bomo modelirali kot krogelno ploskev z določenim lomnim količnikom n in polmerom r . Z uporabo metode "ray tracing", bomo simuliral prehod svetlobnih žarkov skozi tako lečo. Uporabljali bomo program napisan v C++ ter knjižnice za izrisovanje imenovano OpenGL.

2 Fizikalno ozadje

2.1 Snellov zakon

Ko svetlobni žarek prehaja iz medija z lomnim količnikom n_1 v medij z lomnim količnikom n_2 , velja Snellov zakon:

$$n_1 \sin \theta_1 = n_2 \sin \theta_2, \quad (1)$$

kjer je θ_1 vpadni kot in θ_2 lomni kot na normalo ploskve.

2.2 Vektorska oblika Snellovega zakona

Da lahko žarkom sledimo potrebujemo Snellov zakon v vektorski obliki. Poznamo smer vpadnega žarka $\hat{d}$ in normalo ploskve $\hat{n}$, ne pa kotov direktno.

2.2.1 Izpeljava

Označimo:

- $\hat{d}$ - enotski vektor za smer vpadnega žarka,
- $\hat{n}$ - enotski normala ploskve,
- $\hat{t}$ - enotski vektor za smer lomljenega žarka.

Kosinuse kotov glede na normalo dobimo iz skalarnih produktov:

$$\cos \theta_1 = -\hat{d} \cdot \hat{n}, \quad \cos \theta_2 = \sqrt{1 - \sin^2 \theta_1}. \quad (2)$$

Iz kvadrata Snellovega zakona (1) sledi:

$$\sin^2 \theta_2 = \left(\frac{n_1}{n_2}\right)^2 \sin^2 \theta_1 = \left(\frac{n_1}{n_2}\right)^2 (1 - \cos^2 \theta_1). \quad (3)$$

Žarek $\hat{t}$ razstavimo po komponentah glede na ploskev:

$$\hat{t} = \hat{t}_{\parallel} + \hat{t}_{\perp}. \quad (4)$$

Vzporedna komponenta:

$$\hat{t}_{\parallel} = \frac{n_1}{n_2} \left(\hat{d} + \cos \theta_1 \hat{n} \right). \quad (5)$$

Pravokotna komponenta je vzdolž normale:

$$\hat{t}_{\perp} = -\cos \theta_2 \hat{n}. \quad (6)$$

Skupaj torej:

$$\hat{t} = \frac{n_1}{n_2} \hat{d} + \left(\frac{n_1}{n_2} \cos \theta_1 - \cos \theta_2 \right) \hat{n} \quad (7)$$

kjer je $\cos \theta_1 = -\hat{d} \cdot \hat{n}$ in $\cos \theta_2 = \sqrt{1 - (n_1/n_2)^2 (1 - \cos^2 \theta_1)}$.

2.2.2 Popolni odboj

Kadar je $\sin^2 \theta_2 > 1$, lomljenega žarka ni. Nastopi popolni notranji odboj (ang. *total internal reflection*, TIR). Torej ta člen:

$$\left(\frac{n_1}{n_2} \right)^2 (1 - \cos^2 \theta_1) > 1. \quad (8)$$

V programu žarka ne sledimo naprej.

3 Geometrija sfere in žarka

3.1 Enačba sfere

Sfera s središčem $\vec{c}$ in polmerom R je množica vseh točk $\vec{P}$:

$$|\vec{P} - \vec{c}|^2 = R^2. \quad (9)$$

3.2 Presečišče žarka s sfero

Žarek opišemo z enačbo premice:

$$\vec{P}(t) = \vec{o} + t \hat{d}, \quad (10)$$

kjer je $\vec{o}$ izhodišče žarka in $\hat{d}$ enotska smer. Točko $P(t)$ si lahko predstavljamo kot žarek, ki napreduje skozi prostor ko povečujemo t . Vstavimo v enačbo sfere:

$$|\vec{o} + t \hat{d} - \vec{c}|^2 = R^2. \quad (11)$$

Označimo $\vec{oc} = \vec{o} - \vec{c}$ in razvijemo:

$$|\hat{d}|^2 t^2 + 2(\hat{d} \cdot \vec{oc}) t + |\vec{oc}|^2 - R^2 = 0 \quad (12)$$

$$t^2 + 2(\hat{d} \cdot \vec{oc}) t + |\vec{oc}|^2 - R^2 = 0, \quad (|\hat{d}|^2 = 1) \quad (13)$$

kar je kvadratna enačba $at^2 + bt + c = 0$ s koeficienti:

$$a = 1, \quad b = 2 \hat{d} \cdot \vec{oc}, \quad c = |\vec{oc}|^2 - R^2. \quad (14)$$

Diskriminanta:

$$D = b^2 - 4c. \quad (15)$$

- Če $D < 0$: žarek ne seka sfere.
- Če $D \geq 0$: dve rešitvi $t_{1,2} = \frac{-b \pm \sqrt{D}}{2}$.

Izberemo najmanjši pozitivni t (najbližje presečišče pred žarkom):

$$t = \min\{t_1, t_2 \mid t > \varepsilon\}, \quad (16)$$

kjer je $\varepsilon > 0$ majhna vrednost, ki prepreči .

3.3 Normala v točki presečišča

Ko dobimo točko presečišča $\vec{P}_{hit} = \vec{o} + t \hat{d}$, je zunanja normala sfere:

$$\hat{n} = \frac{\vec{P}_{hit} - \vec{c}}{|\vec{P}_{hit} - \vec{c}|} = \frac{\vec{P}_{hit} - \vec{c}}{R}. \quad (17)$$

Pri izstopu iz sfere (druga površina) moramo normalo obrniti: $\hat{n} \rightarrow -\hat{n}$, da pravilno označimo prehod iz gostejšega v redkejši medij.

4 Opis programa

4.1 OpenGL in shaderji

OpenGL je knjižnica narejena v C, ki nam dovoli enostavnejšo sporazumevanje z grafično procesno enoto (ang. *Graphics Processing Unit*, GPU). Koda za inicializacijo oken, bufferjev, resolucije, IO itd. je pretežna, zato ne bo opisana v tej seminarski nalogi. Glavni del pri programiranju grafike je koncept *shaderjev*. To so pod-programi napisani v jeziku GLSL (zelo podoben C), ki se zaženejo za vsak piksel na ekranu vzporedno (so neodvisni en od drugega). To si lahko predstavljamo, kot da ima vsak piksel svoj programček. Vsak ta programček dobi spremenljivko UV — 2D koordinata piksla z vrednostjo med 0 in 1, ki nam pove kje se naš piksel nahaja. GLSL program lahko sprejema tudi druge spremenljivke iz starševskega (main) programa. Takšno spremenljivko imenujemo "uniform".

4.2 Compute shader

V našem primeru uporabljamo poseben tip shaderja imenovan *compute shader*, ki ne riše direktno na zaslon, ampak piše rezultat v buffer (sliko v pomnilniku GPU). Na koncu jo lahko shrani v datoteko ali pa jo izriše na zaslon.

4.3 Metanje žarkov

Iz vsakega piksla "mečemo" svetlobni žarek, ki mu sledimo do ravnine slike, kjer dobi svojo barvo. Izhodišče žarka izračunamo iz UV koordinat:

$$\vec{o} = (UV_x \cdot w, UV_y \cdot h, 0),$$

kjer sta w in h dimenziji zaslona v naši sceni. Smer žarka je za vse piksele enaka (vzporedni žarki):

$$\hat{d} = (0, 0, 1).$$

4.4 Zgradba scene

V naši sceni imamo tri elemente: sliko, lečo in zaslon. Vzporedni žarki simulirajo neskončno oddaljen zaslon.

Pot žarka v realnosti gre od vira svetlobe, zadane sliko, se lomi skozi lečo ter pristane na zaslonu. Pri *ray tracingu* pa to delamo v obratni smeri: žarek svojo pot začne pri zaslonu, gre skozi lečo do slike, in šele tam dobi svojo barvo. S tem smo računalniku prihranili računanje vseh žarkov, ki sploh ne bi zadeli zaslona.

Opazimo tudi, da za našo demonstracijo ne potrebujemo vira svetlobe kot je luč, žarek preprosto prevzame barvo slike, kjer jo zadane.

5 Implementacija v GLSL

5.1 Compute shader — presečišče s sfero

Listing 1: Presečišče žarka s sfero v GLSL

```
float intersectSphere(vec3 origin, vec3 dir, vec3 center, float r) {
    vec3 oc = origin - center;
    float b = 2.0 * dot(dir, oc);
    float c = dot(oc, oc) - r * r;
    float D = b * b - 4.0 * c;
    if (D < 0.0) return -1.0;
    float t1 = (-b - sqrt(D)) / 2.0;
    float t2 = (-b + sqrt(D)) / 2.0;
    if (t1 > 0.001) return t1;
    if (t2 > 0.001) return t2;
    return -1.0;
}
```

5.2 Compute shader — vektorski lom

Implementacija enačbe (7):

Listing 2: Vektorski Snellov zakon v GLSL

```
vec3 refractRay(vec3 d, vec3 normal, float n1, float n2) {
    float cosI = -dot(normal, d);
    float sinT2 = (n1/n2)*(n1/n2) * (1.0 - cosI*cosI);
    if (sinT2 > 1.0) return vec3(0.0); // popolni odboj
    float cosT = sqrt(1.0 - sinT2);
    return (n1/n2)*d + (n1/n2*cosI - cosT)*normal;
}
```

5.3 Potek žarka skozi lečo

Vsak žarek preide skozi dve površini sfere — vstopno in izstopno. Na vsaki površini lom (ponovimo 2x):

Listing 3: Zanka dveh prehodov skozi sfero

```
for (int i = 0; i < 2; i++) {
    float t = intersectSphere(rayOrigin, rayDir, lensCenter, u_r);
    ;
    if (t <= 0.0) break;

    vec3 hit      = rayOrigin + t * rayDir;
    vec3 normal = normalize(hit - lensCenter);
    if (i == 1) normal = -normal; // normala obrnjena za lom
    steklo -> zrak

    float n1 = (i == 0) ? 1.0 : u_n;
    float n2 = (i == 0) ? u_n : 1.0;

    rayDir      = refractRay(rayDir, normal, n1, n2);
    rayOrigin = hit + rayDir * 0.001; // epsilon odmik
}
```

Na prvi površini ($i = 0$) gre žarek iz zraka ($n_1 = 1$) v steklo ($n_2 = n$), na drugi ($i = 1$) pa iz stekla ($n_1 = n$) nazaj v zrak ($n_2 = 1$). Normala na izstopu je obrnjena, ker mora pri vektorski obliki Snellovega zakona normala kazati *stran od medija, v katerem je vpadni žarek*.

5.4 Vzorčenje slike na zaslonu

Po izstopu iz leče žarek propagiramo do ravnine slike na razdalji $u_b + u_a$:

Listing 4: Vzorčenje objektne ravnine

```
//slika na razdalji u_b + u_a
float t_plate = (objectPos.z - rayOrigin.z) / rayDir.z;
vec3 screenHit = t_plate * rayDir + rayOrigin;

vec3 bottomLeftImageBoundry = vec3(objectPos);
vec3 topRightImageBoundry = vec3(objectPos + u_ObjScale);

if(...screen hit...)
color = texture(u_input, (screenHit.xy - objectPos.xy)/
    u_ObjScale);
//za zadje
else color = vec4(0.2*normalize(screenHit.xyy), 1.0);
//za popolni odboj
if(rayDir == vec3(0.0)) {
    color = vec4(0.0,0.0,0.0,1.0);
}
imageStore(outputImage, coord, color);
```

6 Parametri programa

Program sprejema naslednje uniforme, ki jih med izvajanjem nastavljamo z bližnjicami:

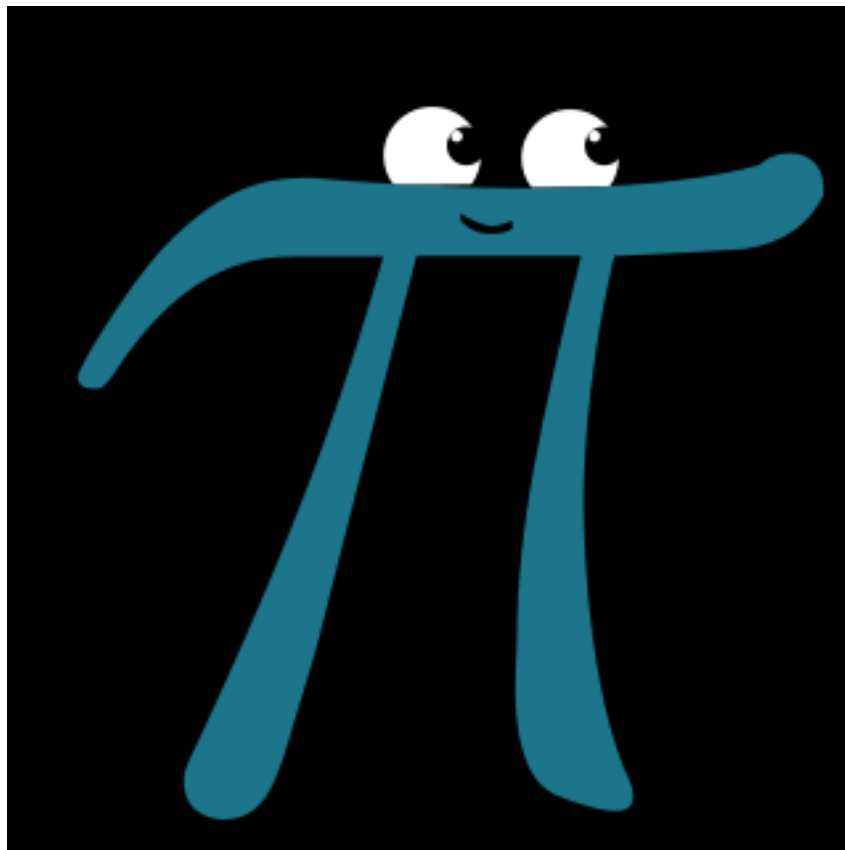
Uniform	Pomen	Tippična vrednost
u_r	Polmer sferne leče	0.0 ... 5.0
u_n	Lomni količnik stekla	1.2
u_a	Razdalja predmeta od leče	10.0 (vzporedni žarki $\rightarrow \infty$)
u_b	Razdalja slike od leče	nastavljivo
u_xLensPos	Horizontalni položaj leče	nastavljivo
u_yLensPos	Vertikalni položaj leče	nastavljivo
u_xObjPos	Horizontalni položaj objekta	nastavljivo
u_yObjPos	Vertikalni položaj objekta	nastavljivo
u_objScale	Velikost objekta na zaslonu	0.0 ... 2.0

Table 1: Uniformni parametri compute shaderja

7 Rezutati

7.1 Testne slike

Testna slika, ki je bila uporabljena:



Slika 1: Testna slika

Ozadje, ki je bilo uporabljeno:



Slika 2: Ozadje

7.2 Prvi render

Zrcaljenje in pomanjšanje slike.

Listing 5: Parametri

```
u_r      = 0.500
u_n      = 1.200
u_a      = 10.000
u_b      = 2.820
u_xLensPos= 0.000
u_yLensPos= 0.000
u_xObjPos = 0.000
u_yObjPos = 0.000
u_ObjScale= 1.000
```



Slika 3: Render 1

7.3 Drugi render

Premaknjena slika.

Listing 6: Parametri

```
u_r      = 0.330
u_n      = 1.200
u_a      = 10.000
u_b      = 2.220
u_xLensPos= 0.000
u_yLensPos= 0.000
u_xObjPos = 0.030
u_yObjPos = 0.340
u_objScale= 0.620
```



Slika 4: Render 2

7.4 Tretji render

Manjša leča, bolj popačena slika.

Listing 7: Parametri

```
u_r      = 0.550
u_n      = 1.200
u_a      = 10.000
u_b      = 2.750
u_xLensPos= 0.000
u_yLensPos= 0.000
u_xObjPos = 0.000
u_yObjPos = 0.000
u_ObjScale= 1.000
```



Slika 5: Render 3

7.5 Četrti render

Večja leča, manj popačena slika.

Listing 8: Parametri

```
u_r      = 1.290
u_n      = 1.200
u_a      = 10.000
u_b      = 7.650
u_xLensPos= 0.000
u_yLensPos= 0.000
u_xObjPos = 0.000
u_yObjPos = 0.000
u_ObjScale= 1.000
```



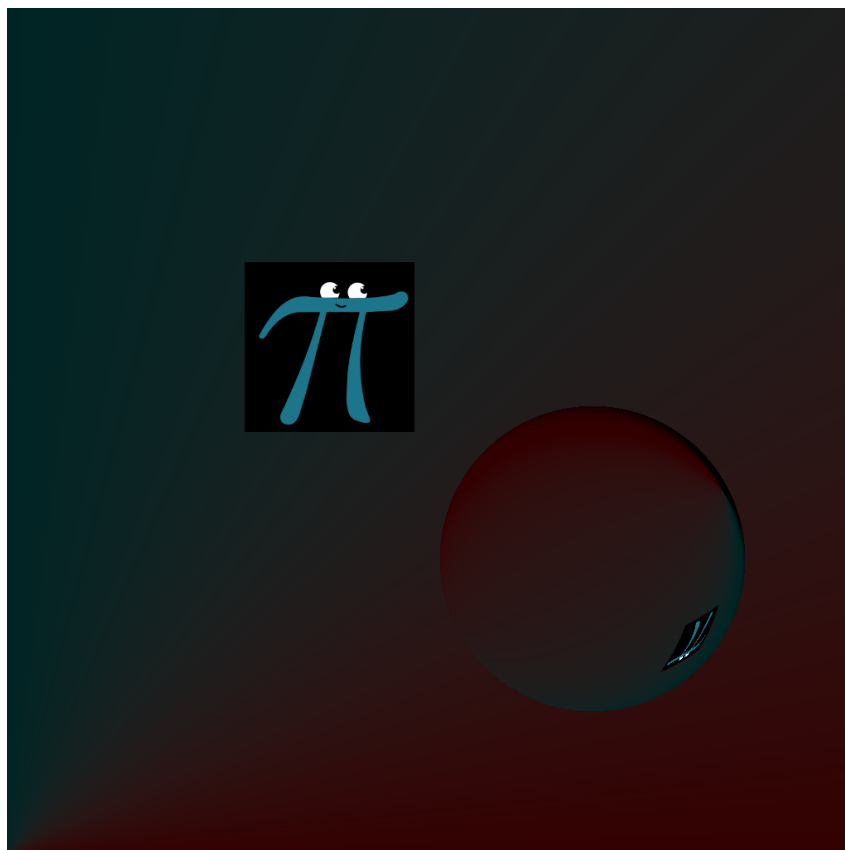
Slika 6: Render 4

7.6 Peti render

Leča ne pokriva slike.

Listing 9: Parametri

```
u_r      = 0.180
u_n      = 1.200
u_a      = 10.000
u_b      = 1.120
u_xLensPos= 0.190
u_yLensPos= -0.150
u_xObjPos = 0.280
u_yObjPos = 0.500
u_ObjScale= 0.200
```



Slika 7: Render 5

8 Zaključek

V tej seminarski nalogi smo spoznali metodo "ray tracing" za simulacijo sferične aberacije skozi sferično lečo.

Pokazali smo, da lahko Snellov zakon zapišemo v vektorski obliki, ki je neposredno uporabna v računalniški grafiki — brez eksplicitnega računanja kotov. Presečišče žarka s sfero smo izpeljali kot kvadratno enačbo, katere rešitvi določata vstopno in izstopno točko na leči.

Iz rezultatov je razvidno, da manjši polmer leče r povzroči večjo sferično aberacijo — žarki, ki potujejo skozi rob leče, se lomijo bistveno bolj kot paraksialni žarki blizu osi, kar se kaže kot zamegljena in popačena slika. Pri večjem r se žarki zberejo bližje skupaj in slika je ostrejša.

Čeprav je izrisovan ekran bil velik $1024 \times 1024 = 1'048'576$, to pomeni več kot 1 milijon žarkov na sliko je GPU (v našem primeru NVIDIA GeForce RTX 3060 Ti) sliko v povprečju generiral le v 6 milisekundah! Kar nam pove kako učinkovito je vzporedno procesiranje za takšne naloge.

9 Viri

P. Shirley, *Ray Tracing in One Weekend*, 2020.

Dostopno na: <https://raytracing.github.io/>

Khronos Group, *GLSL Specification 4.60*.

Dostopno na:

<https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf>

Khronos Group, *OpenGL 4.3 Core Profile Specification*, 2012.

Dostopno na:

<https://registry.khronos.org/OpenGL/specs/gl/glspec43.core.pdf>